Комьюнити учеников Яндекс Лицея!
Мероприятия, турниры, конкурсы, новые знакомства — ждём тебя 😎

Подробнее

Как спастись из матрёшки циклов и пошпионить за кодом

- Переменные как флаги
- Прерывание цикла
- Else-часть у циклов
- Вложенные циклы
- Выход из цикла
- Отладка программ

Мощь циклов `for` и `while` позволяет реализовывать достаточно сложные алгоритмы. Но их потенциал ещё раскрыт не до конца. Сегодня вы узнаете, как вырваться из плена циклов, пропустить скучные итерации и освоить вложенные миры.

А ещё познакомитесь с очень важным занятием, за которым программисты проводят больше всего времени (и это не написание кода и даже не рабочие созвоны).

🏁 Поднять булев флаг

Мы уже говорили про использование булевых переменных как флагов, но вскользь. А теперь давайте ещё раз, но более конкретно.

Представьте, что вы охотник за багами в старой системе управления тостерами. У вас есть кнопка, которую нельзя нажимать. Но пользователи — такие пользователи. Поэтому вы ставите флажок — булеву переменную `was_pressed`, которая **сначала** `False`, а если кнопку нажали, становится `True`. Всё. Это флаг.

```
    was_pressed = True
```

Чаще всего схема с флагами используется в комбинации с циклом `while`, где флажок является единственным условием завершения цикла:

```
ready_to_exit = False
while not ready_to_exit:
    command = input()
    if command == 'exit':
        ready_to_exit = True
    ... # Какой-то полезный код далее
```

Либо является дополнительным «пожарным выходом»:

```
need_emergency_exit = False
check_sum = 0
while check_sum < 1000 and not need_emergency_exit:  # Запасной выход дол
    command = input()
    if command == 'exit':
        need_emergency_exit = True
    ... # Какой-то полезный код далее
```

Но никто не может нам запретить использовать флаг и с циклом `for`, хотя тут смысла меньше, так как сам по себе флаг в этом случае никак не повлияет на количество итераций цикла. Мы всё равно дойдём до конца перебора и только потом узнаем, случилось ли целевое событие.

В примере ниже (эта программа — терапевтический тренажёр для избавления физиков-экспериментаторов от синхрофазотронозависимости) имеется флаг `said_forbidden_word`, который означает «сказал ли пользователь запретное слово „синхрофазотрон“». Флаг равен `True`, если сказал, и `False`, если нет.

```
forbidden_word = 'синхрофазотрон'
print('Введите десять слов, но постарайтесь случайно не ввести слово "' +
      forbidden_word + '"!')
# Можно было использовать и sep='', чтобы кавычки не отклеились от слова
said_forbidden_word = False
for i in range(10):
```

```
    word = input()
    if word == forbidden_word:
        said_forbidden_word = True  # Но цикл дойдём до конца
    # Вместо предыдущих двух строк также можно написать:
    # said_forbidden_word = (said_forbidden_word or word == forbidden_wor
if said_forbidden_word:
    print('Вы нарушили инструкции.')
else:
    print('Спасибо, что ни разу не упомянули', forbidden_word)
```

В самом начале пользователь ещё ничего не успел сказать, поэтому флаг установлен в `False`. Далее на каждой итерации цикла, если пользователь сказал запретное слово, флаг устанавливается в `True` и остаётся в таком состоянии (при необходимости флаг можно и «опустить»). Как только флаг оказывается равен `True`, поведение программы меняется: перед каждым вводом выдаётся предупреждение, а в конце выдаётся другое сообщение.

Давайте переменным-флагам осмысленные имена. Принято, что они должны

🚪 Выходим красиво

Но что, если нам и в цикле `for`, и в цикле `while` нужно выйти, не дожидаясь конца текущей итерации, прямо в тот момент, когда мы осознали, что пора? Такая возможность есть.

Оператор `break` — это ваш личный телепорт из цикла. Он мгновенно переносит вас за его пределы, даже если итерации не закончились.

```
while True:
    action = input('Что делать? (стоп/продолжить): ')
    if action == 'стоп':
        print('Сбежали!')
        break
    print('Делаем:', action)
print('Бесконечный цикл дошёл до конца')
```

И с `for`:

```
        print(f"Первое чётное: {num}")
        break
print(f"Закончили цикл на числе {num}")
```

break — это когда вы смотрите сериал, а после второй серии третьего сезона понимаете, что сценарий скатился. Хлоп — и вы закрываете «Кинопоиск». Будьте внимательны, break — штука мощная. Если использовать её бездумно, код станет хаотичным и непредсказуемым.

Сравним одну и ту же программу с флагом:

```
count = 0
need_exit = False
while not need_exit:
    command = input("Введите 'e' для выхода: ")
    if command == 'e':
        need_exit = True
    else:
        count += 1
        print(f"Вы зашли в цикл {count} раз.")
```

И с break:

```
count = 0
while True:
    command = input("Введите 'e' для выхода: ")
    if command == 'e':
        break
    count += 1
    print(f"Вы зашли в цикл {count} раз.")
```

⏭️ Эта итерация нам не идёт

А если мы хотим не совсем-совсем выходить из цикла, а только перейти к следующей итерации? И такое можно.

Оператор continue — это как сказать «пока-пока» текущей итерации и сразу перепрыгнуть к заголовку цикла (для проверки условия или для получения

```python
for i in range(6):
    if i == 2:
        continue  # Испытываем личную неприязнь к двойкам
    print(i)
```

Использовать `continue` в цикле `while` надо с осторожностью, чтобы избежать ситуации, когда мы из-за пропуска итерации перестаём изменять переменную, благодаря которой мы должны когда-нибудь выйти из цикла:

```python
count = 1
while count < 100:
    if count % 5 == 0:
        continue  # Мы не поменяли счётчик и обречены на бесконечное повт
    print(count)
    count += 1
```

Оператор `continue` — это пропустить филлерную серию и продолжить смотреть сериал дальше.

Используйте `continue`, чтобы избегать вложенных `if`. Код становится чище:

```python
# Плохо
for item in items:
    if условие:
        # 10 строк кода

# Хорошо
for item in items:
    if не_условие:
        continue
    # 10 строк кода
```

🤔 А иначе что?

Как мы знаем, у `if` есть необязательная часть `else`. Так вот у циклов она тоже есть!

Блок `else` у цикла выполняется, если он завершился без `break`.

Пример с `for`:

```
print('Предъявите число, я проверю, простое ли оно.')
num = int(input())
for i in range(2, num):  # Не идеальный range, подумайте, как лучше
    if num % i == 0:
        print(f"{num} не простое.")
        break
else:
    print(f"{num} простое!")  # Сработает, если делителей нет
```

Пример с `while`:

```
password = "secret"
attempts = 3
while attempts > 0:
    guess = input("Введите пароль: ")
    if guess == password:
        print("Доступ разрешён!")
        break
    attempts -= 1
else:  # Если закончились попытки
    print("Доступ запрещён. Вызываю полицию!")
```

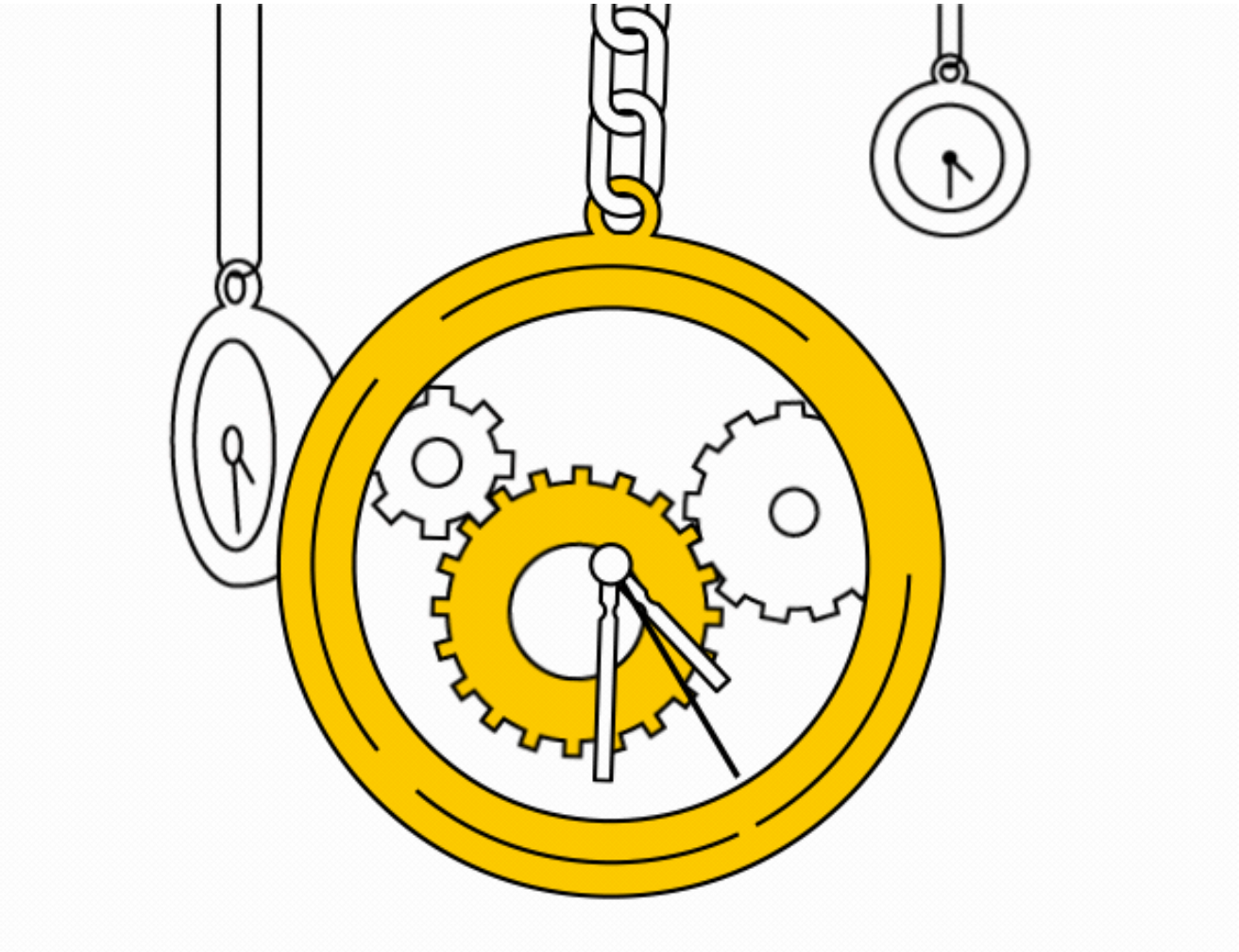
Если `break` — это завершение просмотра сериала посреди сезона, а `continue` — пропуск серии, то `else` в цикле — финальные титры в конце.

Можно выполнять цикл, пока выполняешь цикл

Мы уже упоминали, что Python не терпит никакой дискриминации, и можно бесконечно вкладывать одни блочные операторы в другие. Это означает, что никто не может помешать нам засунуть цикл в цикл в цикле и т. д.

Тот цикл, который находится на более верхнем уровне (с меньшим уровнем вложенности), называют **внешним**, а тот, что внутри, — **вложенным**.

Каждую итерацию внешнего цикла внутренний выполняется полностью. Самый простой пример из жизни — аналоговые часы. Каждые сутки — одну итерацию часовой стрелки — выполняется 24 итерации минутной стрелки и 24 * 60 (1 440) итераций

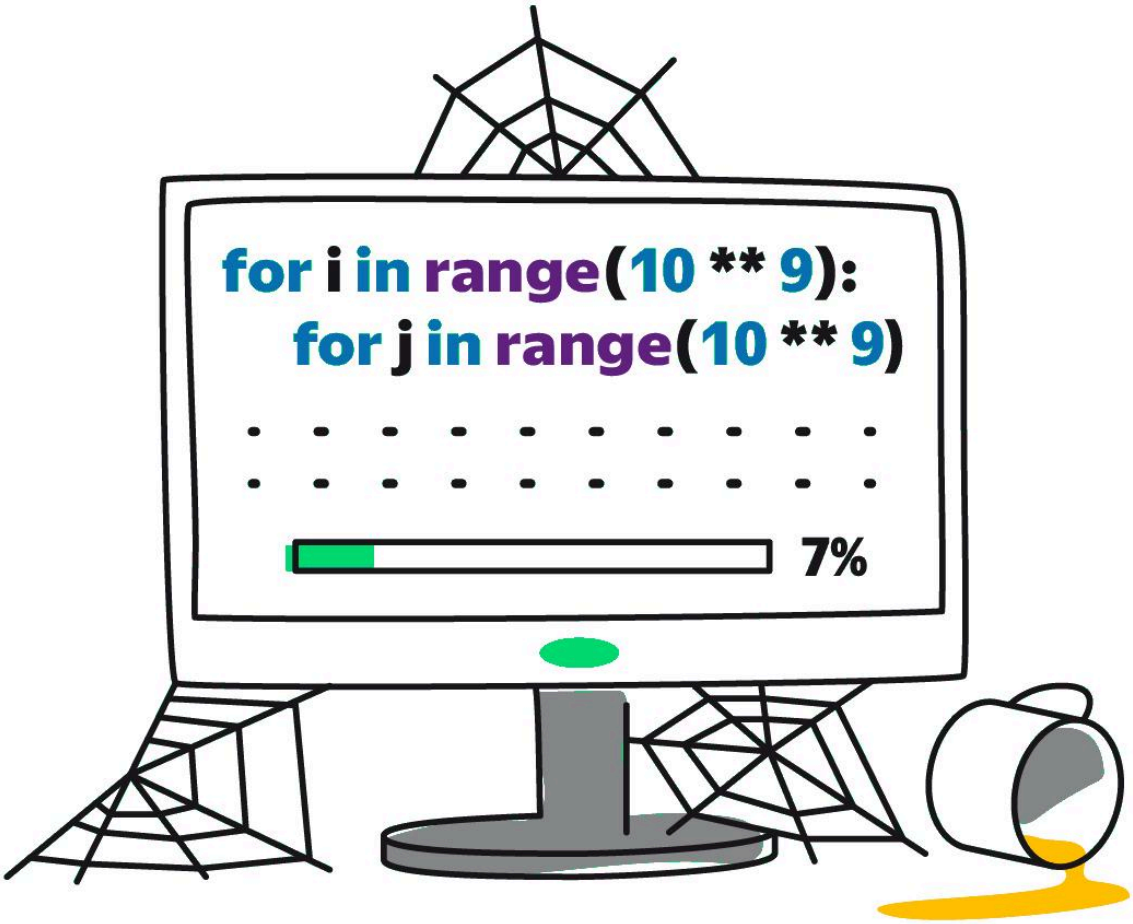


Если говорить языком Python, то это будет выглядеть вот так:

```
for h in range(24):  # Одни сутки — 24 часа
    for m in range(60):  # В каждом часе 60 минут
        for s in range(60):  # В каждой минуте 60 секунд
            print(f"Итерация внутреннего цикла: {(h + 1) * (m + 1) * (s +
            print(f"Время: {h:02d}:{m:02d}:{s:02d}")
```

Вложенные циклы позволяют составлять более сложные алгоритмы, но если их становится слишком много, то вы рискуете потеряться в измерениях.

К тому же вложенные циклы имеют сложность **O(n²)**. Если внешний цикл выполняется 1000 раз, а внутренний — тоже 1000, всего итераций будет 1_000_000. Это если вложенный цикл один. Если есть вложенность третьего уровня, то будет уже третья степень и 1_000_000_000, и так далее. Это как пытаться пересчитать все песчинки на пляже — то есть **медленно**.



Да, практически всегда в учебных задачах это «медленно, но достаточно», хотя иногда (не в этом уроке) вам будут встречаться задачи, где надо будет придумать более хитрый алгоритм, чтобы избежать вложенных циклов, иначе программа будет выполняться непозволительно долго.

Давайте рассмотрим ещё один пример. Необходимо вывести в строку таблицу умножения для заданного числа. Задача решается так:

```
k = int(input())
for i in range(1, 10):
    print(f"{i} * {k} = {k * i}", end='\t')
```

Очевидно, что в этом случае предыдущую программу нужно повторить k раз, где вместо k будут использоваться числа от 1 до k включительно.

Эту задачу можно записать двумя циклами, где для каждого значения внешнего цикла будут выполняться все значения внутреннего цикла.

Программа будет выглядеть так:

```
k = int(input())
for j in range(1, k + 1):
    for i in range(1, 10):
        print(f"{i} * {j} = {j * i}", end='\t')
    print()  # Пустая строка после всех итераций внутреннего цикла
```

Смотрите, тут два слоя циклов. Внешний со счётчиком `j` крутится от 1 до k (включительно). **Что происходит по шагам**:

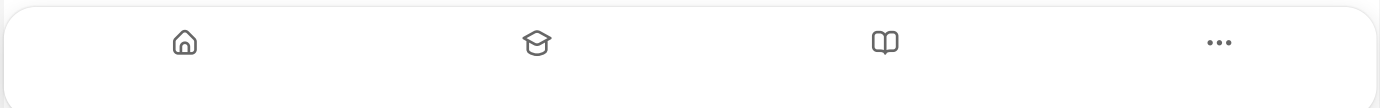
1. **Внешний цикл:**
   - Стартуем с `j = 1`.
2. **Внутренний цикл (i):**
   - `i` бегает от 1 до 9 (потому что `range(1, 10)` даёт 1, 2, ..., 9).
   - На каждой итерации: печатаем `i * j` через форматированный вывод (сепараторы, табы — красота!).
   - Когда `i` добирает до `10` → внутренний цикл завершается.
   - Делаем `print()` для перевода строки.
3. **Возврат во внешний цикл:**
   - `j` увеличивается на 1.
   - Если да — снова ныряем во внутренний цикл.

Фишка вложенности ещё раз: Пока внутренний цикл не отработает **все** итерации при текущем `j`, внешний цикл не сдвинется.

🚀 Катапультирование из любой глубины

Операторы `break` и `continue` классные, но они работают **только непосредственно в том цикле, в котором находятся**. Но что делать, если нам надо вырваться из второго уровня вложенности? Есть несколько способов.

1. Один `break` хорошо, а в комбинации с флагом — ещё лучше:



```
runaway = False
```

```
for j in range(1, 100):
    for i in range(1, 100):
        if i * j > 700:
            print('A-a-a-a-a-a, попалось слишком большое число!')
            runaway = True  # Ставим флаг
            break  # Выходим из вложенного цикла
        print(i ** j)
    if runaway:
        break  # Выходим из внешнего цикла
```

И... это пока единственный способ, доступный нам на текущий момент. Ещё как минимум с двумя другими — выходом из функции и использованием исключений — мы познакомимся несколько позже.

🔎 Как искать чёрную кошку в тёмной комнате, а потом понять, что это был енот

Пока наши программы не слишком большие, но они уже достигли такого уровня сложности, что поиск логической ошибки в коде начинает быть похожим на игру в «Морской бой» — мы исправляем строчки наугад в надежде, что вот это-то точно поможет. И дальше будет только сложнее. Поэтому пора нам познакомиться с тем, как вообще правильно искать ошибки (баги) в коде, то есть заниматься отладкой или дебаггингом.

Термин «дебаггинг» (от англ. debug — «удалять жуков») — это не просто метафора, а реальная история с насекомым, ставшая IT-легендой.
В 1947 году программистка Грейс Хоппер работала с компьютером `Harvard Mark II`. Внезапно машина начала глючить, и после вскрытия обнаружили... зажатого в реле мотылька. Насекомое записали в журнал с пометкой: «First actual case of bug being found» («Первый реальный случай обнаружения жука»). Позже слово **debug** стало означать «поиск и устранение ошибок». Мотылёк даже попал в музей — его до сих пор можно увидеть в стеклянной колбе.
Само слово bug (жук, насекомое) как метафора для неисправности использовалось ещё в XIX веке! Томас Эдисон писал про «жуков» в в своих изобретениях ещё в 1878 году, а в телеграфии так называли помехи в сигнале.
То есть инженеры давно «ловили жуков» — просто Грейс Хоппер визуализировала термин буквально.

Рассмотрим пример. Нам надо написать программу, которая заставляет пользователя

числа с помощью операции получения остатка от деления на 10. У нас получится какой-то вот такой код:

```
print('Тренажёр по вводу палиндрома:')
while True:
    print('Введите число-палиндром:')
    number = n = int(input())
    reverse = 0
    while n > 0:
        reverse = reverse * 10 + n % 10
        n //= 10
    if number == reverse:
        print('Вы ввели палиндром! Программа остановлена.')
        break
    print('Введённое число не палиндром, попробуйте ещё раз.')
```

Самый базовый путь «отладки» — навтыкать `print()` в стратегические места, чтобы переменные пели, как канарейки. Например, внутри внутреннего цикла выводим, как меняется `n` и `reverse`:

```
while n > 0:
    reverse = reverse * 10 + n % 10
    n //= 10
    print(f"n: {n}, reverse: {reverse} → Может, уже сдадитесь, палиндромы
```

Этот способ может рассматриваться как рабочий в нескольких случаях:

- Программа небольшая (но не забывайте удалить такие отладочные принты перед сдачей, а то проверяющая система взбунтуется).
- Ваша программа работает на каком-то удалённом сервере, и ошибки получаете даже не вы, а неизвестные пользователи вашего программного обеспечения. Тут логи (более сложный вариант отладочных принтов) вообще единственный способ разобраться, что же пошло не так.

Поэтому давайте посмотрим, как происходит отладка по-взрослому:

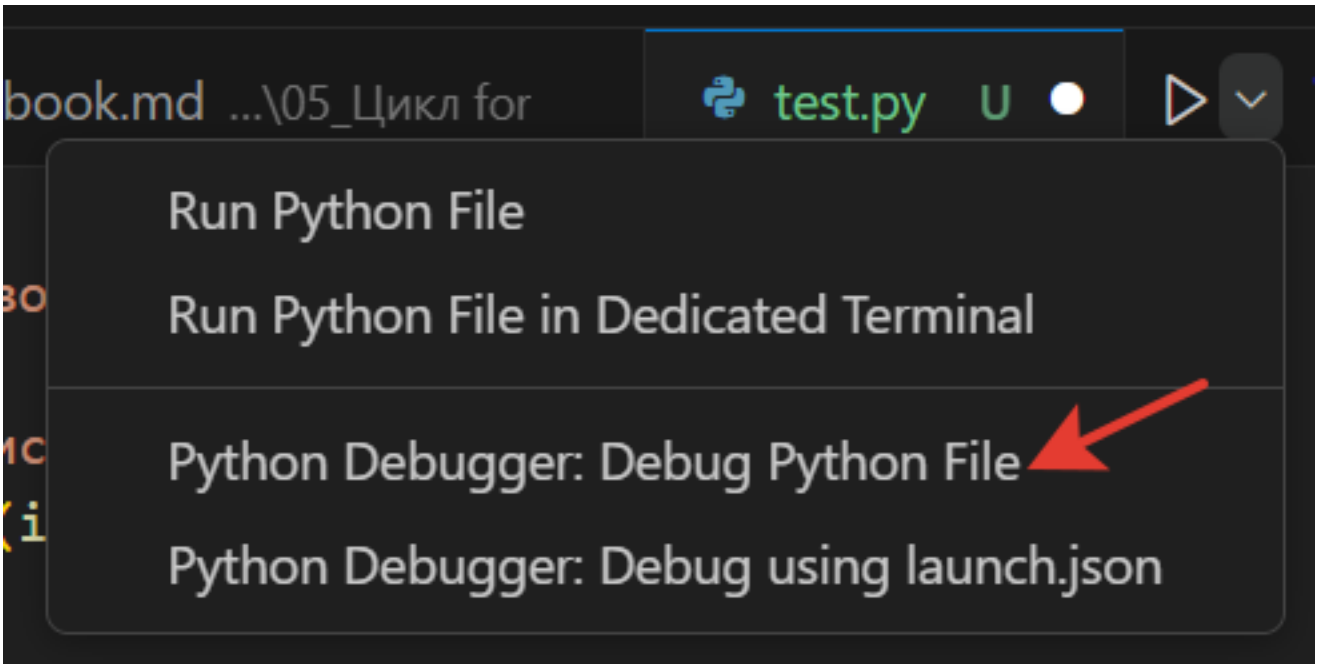
Инструкция:

1. Ставим ловушки (точки останова или брейкпоинты) в потенциально интересных

- На строке `number = n = int(input())` → чтобы поймать момент ввода.
- Внутри внутреннего цикла → следим, как `reverse` растёт, как гриб после дождя.
- На `if number == reverse` → проверяем финальный аккорд.

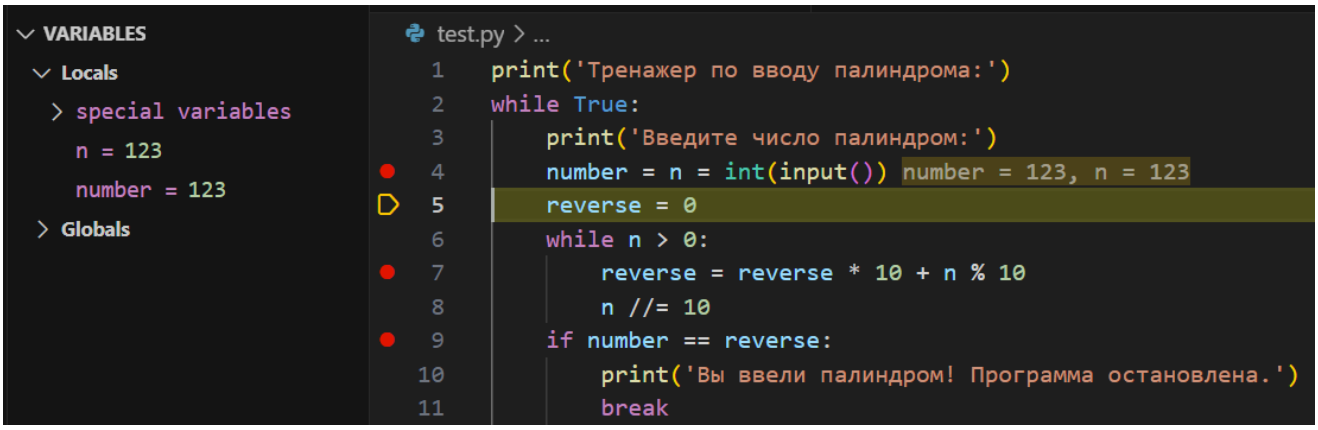
```python
print('Тренажер по вводу палиндрома:')
while True:
    print('Введите число палиндром:')
    number = n = int(input())
    reverse = 0
    while n > 0:
        reverse = reverse * 10 + n % 10
        n //= 10
    if number == reverse:
        print('Вы ввели палиндром! Программа остановлена.')
        break
    print('Введенное число не палиндром, попробуйте еще раз.')
```

2. Запускаем дебаггер.



3. Вводим число:

- Допустим, вводим 123.
- Программа останавливается на первом брейкпоинте: смотрим, что `number = 123`, `n = 123`.
- Шагаем (горячая клавиша `F10`) через внутренний цикл и смотрим, как меняются значения переменных:



Шаг 1: `reverse = 0 * 10 + 123 % 10 = 3`, `n = 123 // 10 = 12`.

Шаг 2: `reverse = 3 * 10 + 12 % 10 = 32`, `n = 12 // 10 = 1`.

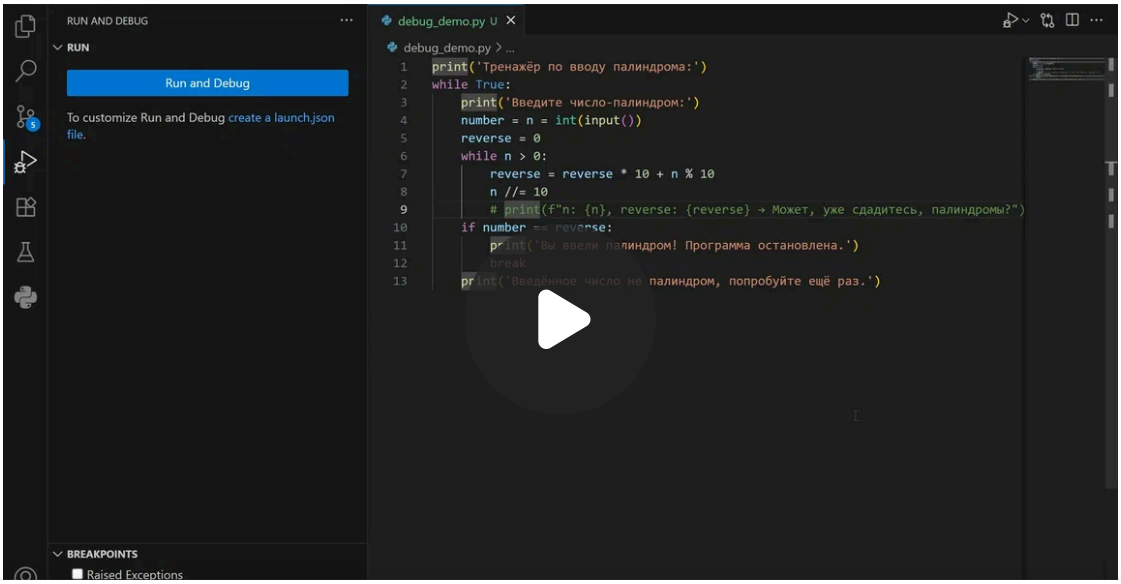
Шаг 3: `reverse = 32 * 10 + 1 % 10 = 321`, `n = 1 // 10 = 0`.

Цикл завершается. Смотрим: `reverse = 321`, `number = 123` → условие не срабатывает.

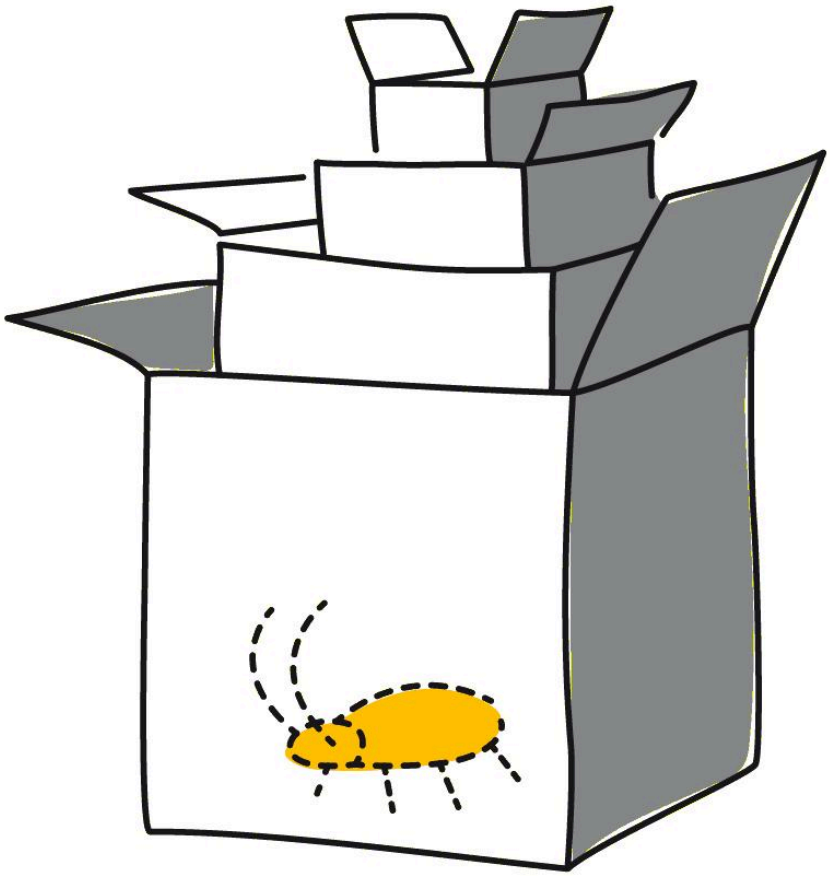
Вывод: Всё работает, но палиндрома нет.

- Проверяем крайние случаи: Вводим 121 → дебаггер показывает, как `reverse` становится 121 → условие срабатывает.
- Вводим 5 → внутренний цикл не выполняется (`n = 5 > 0` → выполняется один раз: `reverse = 5`, `n = 0`). Условие `5 == 5` → палиндром!

При прочтении может быть не очень понятно, что вообще происходит, поэтому рекомендуем посмотреть видео (поговорка про лучше один раз увидеть не на пустом месте появилась).



Попробуйте повводить разные числа и посмотреть как ведёт себя программа. Попробуйте найти баг (спойлер: программа странно работает для отрицательных чисел, но, возможно, это не баг, а фича).



🌟 Итог: знаем о циклах всё (всё?)

Чисто теоретически с использованием текущих знаний вы можете решить любую задачу в мире*. Ведь сегодня вы узнали:

- Что `break` и `continue` — ваши союзники для управления ходом исполнения циклом, но важно использовать их с умом, чтобы не превратить программу в визуализацию теории заговора.
- Флаги делают код понятнее (но превращать код в ёлку из флажков тоже так себе

- Вложенные циклы — мощь, которая требует аккуратности и убивает производительность, если входные данные очень большие. Если вложенность больше 3 — скорее всего, вы что-то делаете не так.
- Отладка позволяет избавиться от ошибок и снижает требования к разработчику в плане ясновидения.

* С некоторыми оговорками — вам иногда нужно будет заводить тысячи переменных, писать одни и те же куски кода много сотен раз (и много-много грустить).

P. S. Если вам кажется, что ваш код работает правильно с первого раза, то алгоритм действий такой:

1. Убедиться, что вы не спите.

2. Проверить программу на крайних случаях.

И только потом радоваться.